A classification of well-behaved graph clustering schemes

Vilhelm Agdur

February 26, 2025

Abstract

Community detection in graphs is a problem that is likely to be relevant whenever network data appears, and consequently the problem has received much attention with many different methods and algorithms applied. However, many of these methods are hard to study theoretically, and they optimise for somewhat different goals. A general and rigorous account of the problem and possible methods remains elusive.

We study the class of all clustering methods that are monotone under addition of vertices and edges, phrasing this as a functoriality notion. We show that if additionally we require the methods to have no resolution limit in a strong sense, this is equivalent to a notion of representability, which requires them to be explainable and determined by a representing set of graphs. We show that representable clustering methods are always computable in polynomial time, and in any nowhere dense class they are computable in roughly quadratic time.

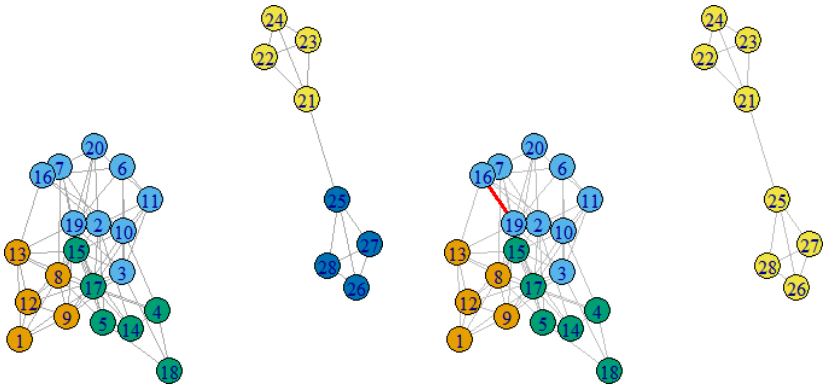
Finally, we extend our definitions to the case of hierarchical clustering, and give a notion of representability for hierarchical clustering schemes.

1 Introduction

The problem of inferring community structure from a graph is one that appears in many different guises in different research areas. It is useful for everything from assigning research papers to subfields of physics [CR10], assessing the development of nano-medicine by studying patent cocitation networks [BAB12], studying brain function via the community structure of the connectome [Bet23], understanding how different bird species spread various plant seeds [Cos+16], to studying hate speech on Twitter [Evk+21].

Despite, or perhaps because of, the widespread nature of this problem, there is no universally agreed on definition of what is actually meant by community structure. This leads to varying methods being used to solve the problem, that do not necessarily give the same results. One of the most popular definitions in practice is the modularity of a partition, introduced by Newman and Girvan [NG04] – it is the method used in all the examples given in the previous paragraph. There are however many other methods, such as the flow-based infomap method motivated by information theory [RAB09], more statistically sound methods assuming a generative model [Pei14], novel graph neural network methods [Koj+23], and many more [Jav+18].

One thing nearly all of these methods have in common is that they are in a sense black boxes – they do not give you any way of answering the question of *why* two vertices were put in the same part. They are also generally hard to study in a rigorous mathematical way, resulting in few theoretical guarantees for their behaviour. While modularity has proven



(a) A graph G, with vertices coloured according to the modularity-maximising partition

(b) G with a single edge added, highlighted in red, and its new optimal partition

Figure 1: An illustration of the resolution limit issue with modularity.

itself very useful in practice, it is in fact guaranteed to behave in undesirable ways in certain settings.

In particular, modularity suffers from a "resolution limit", wherein it cannot "see" things happening in subgraphs that are small compared to the total graph, resulting in very unexpected behaviour [FB07]. In Figure 1 we see one illustration of this phenomenon – one would hope that what happens in one connected component cannot affect the other, but the resolution limit results in this undesirable behaviour.

Another issue with studying modularity optimization from a theoretical perspective is the fact that computing it is NP-hard [Bra+08]. This means that when using modularity maximization in practice, one always needs a heuristic algorithm for finding good partitions, such as the Louvain [Blo+08] or Leiden [TWV19] algorithms, which themselves do not have many known theoretical guarantees connecting them to modularity. Thus, one is left having to either study modularity itself, and hoping that the results say something about the results of these algorithms, or studying the algorithms themselves, and not directly saying anything about modularity.

These issues with the varying optimization based approaches to clustering lead us to investigating a more "axiomatic" approach to the problem – that is, we want to specify a few properties that the clustering method must have, and then see what methods there are that satisfy these properties. Other than of course giving us guarantees on the behaviour of our methods, this approach also gives us much more straightforward algorithms which run in polynomial time, since we are no longer doing optimization, but in some sense just applying a decision rule for when vertices should be in the same part or not.

Our particular approach to this is motivated by the results of Carlsson and Mémoli [CM13] for the analogous problem of clustering finite metric spaces. Like in their setting, there are three properties that are of interest:

1. *Functoriality* is essentially a form of monotonicity under addition of edges or vertices to the graph. Adding a new edge or new vertex should only ever make communities more closely connected, it should never split them.

2. *Excisiveness* is a very strict version of requiring that there be no resolution limit – instead we require that the method be idempotent on the parts it has found. That is, if we take some graph G, cluster it, and then look at the subgraph induced by any part, that graph has to be clustered into just a single large component, so we never discover new features at smaller scales.

3. *Representability* is a type of explainability of the clustering method. A representable clustering method $\Phi_{\mathfrak{R}}$ is determined by a set $\mathfrak{R}$ of simple graphs, and the rule that two vertices of a graph G will be in the same part if their neighbourhood looks like something in $\mathfrak{R}$. This also gives us a simple algorithm to actually compute the partitioning of a graph, that runs in polynomial time for each fixed $\mathfrak{R}$.

Our main result classifies exactly the relationship between these properties, and shows that all representable clustering schemes are tractable on a large class of sparse graphs:

Theorem 1.1 (Main results)**.** *A clustering scheme is representable if and only if it is excisive and functorial. Further, any such representable clustering scheme is computable in quadratic time on any graph class of bounded expansion.*

We also give a structural result for when two representable clustering schemes are equal, and use this to prove that there exist representable clustering schemes that are not finitely representable. Finally, we extend our definition of a representable clustering scheme into the setting of hierarchical clustering schemes. Here, instead of just assigning a single clustering to each graph, we give one clustering for each "scale", giving a spectrum of clusterings of the graph from coarsest to finest. In this setting, we find that the connection to excisiveness no longer works.

2 Notation

1. We use capital Roman letters for graphs, e.g. G or H, and their vertex and edge sets are also capital Roman letters, e.g. V, W and E, F.

2. We use small Roman letters for vertices or edges of graphs, e.g. v or e.

3. For the function sending an edge to its underlying vertex set, we use the character $\mathfrak{e}$, pronounced "e".

4. For functors, we use capital Greek letters, e.g. Φ, Π, Λ.

5. For clustering schemes of graphs, and also for representing sets for representable clustering schemes, we use Fraktur letters, e.g. $\mathfrak{C}$, $\mathfrak{R}$.

6. For morphisms between graphs we normally use small Roman letters, e.g. f, g, except when considering morphisms from a graph R in a representing set $\mathfrak{R}$, where we instead use small Greek letters, e.g. ω.

7. If $f : X \to Y$ is a set function, we write f_* for the function from 2^X to 2^Y that sends a subset of X to its image under f in Y.

3 Definitions of our terms

Definition 3.1. The category of hypergraphs $\mathcal{G}$ has as its objects hypergraphs, which we consider to be a triple $(V, E, \mathfrak{q})$ of a finite set V of vertices, a finite set E of edges that is disjoint from V, and a function $\mathfrak{q} : E \to 2^V$ assigning each edge its vertex set. Notice that this allows us to have multiple edges with the same edge set. To simplify some technical points, we will assume that all our hypergraphs have loops at every vertex, that is, that for each $v \in V$ there is an $e \in E$ such that $\mathfrak{q}(e) = \{v\}$. Where do we actually need the assumption of loops at every vertex?

A morphism from $H = (V, E, \mathfrak{q})$ to $G = (V', E', \mathfrak{q}')$ is an injective set function $f : V \to V'$ such that, for every $e \in E$, there is an $e' \in E'$ such that $f_*(\mathfrak{q}(e)) = \mathfrak{q}'(e')$. That is, a morphism from H to G is a way of realising H as a sub-hypergraph of G.

Definition 3.2. The category of simple graphs $\mathcal{G}_s$ is the full subcategory of $\mathcal{G}$ whose objects are the simple graphs, that is, the graphs all of whose edges contain either one or two vertices. Again, we allow edges with a single vertex for technical reasons.

Whenever we use just the word "graph", we mean a hypergraph, since those are more common for us.

Definition 3.3. The category $\mathcal{P}$ of sets with overlapping partitions has as its objects finite sets V together with a set cover $P \subseteq 2^V$. That is, we assume that for each $v \in V$ there is at least one $p \in P$ such that $v \in p$. This is where we needed the assumption of loops at every vertex – we could also instead remove both that assumption and the assumption that all vertices are in at least one part.

A morphism in $\mathcal{P}$ from (V, P) to (W, Q) is a set function $f : V \to W$ such that for every $p \in P$, there exists a $q \in Q$ such that $f_*(p) \subseteq q$.

Definition 3.4. The category $\mathcal{P}_{\text{no}}$ of sets with non-overlapping partitions is the full subcategory of $\mathcal{P}$ whose objects have disjoint set covers. We may equivalently think of it as the category of sets equipped with equivalence relations.

Here it is worth remarking that the categories $\mathcal{G}$ and $\mathcal{P}$ are very similar, in that they have the same objects, but we have a smaller set of morphisms among the hypergraphs than among the sets with overlapping partitions. However, for clarity's sake, we shall usually forget this fact, and treat these as two unrelated categories, instead of thinking of them as one contained within the other.

Definition 3.5. An overlapping clustering scheme is a function $\mathfrak{C}$ that sends hypergraphs to sets with overlapping partitions, with the same underlying set – that is, if $\mathfrak{C}$ sends $G = (V, E, \mathfrak{q})$ to $P = (W, P)$, we require that $V = W$. We say that it is *functorial* if it is a functor from $\mathcal{G}$ to $\mathcal{P}$. We will often abuse our notation slightly and write $p \in \mathfrak{C}(G)$ when what we mean is that $\mathfrak{C}(G) = (V, P)$ and $p \in P$.

One good way to think of these definitions is as a kind of generalization of monotonicity. If we were restricting to only studying clustering of graphs on a fixed underlying set, the statement that there is a morphism from G to H becomes precisely the statement that $G \subseteq H$, that is that G is less than H in the subgraph inclusion order. Likewise, if we restricted to only studying non-overlapping partitions, there is a natural partial order on the set of equivalence relations on the underlying vertex set, given by that $\sim < \approx$ if $\sim$ is a refinement of $\approx$, or equivalently if $\approx$ is a coarsening of $\sim$.

In this restricted setting, the statement that a clustering scheme is functorial becomes exactly the statement that it is monotone with respect to the inclusion order and the order on equivalence relations. By using the language of category theory to encode this information, we gain the ability to talk about things being "monotone under addition of vertices", not just under addition of edges, and it becomes easier to speak about monotonicity with respect to non-overlapping set partitions.

Definition 3.6. A clustering scheme $\mathfrak{C}$ is *excisive* if the following property holds for all hypergraphs $G = (V, E, \mathfrak{r}_G)$: If $\mathfrak{C}(G) = (V, P)$, it holds for every part $p \in P$ that $\mathfrak{C}(G|_p) = \mathfrak{C}(G)|_p$. Here, $G|_p$ is the subgraph induced by p, so that the vertex set of $G|_p$ is p, $E(G|_p) = \{e \in E \mid \mathfrak{r}_G(e) \subseteq p\}$, and $\mathfrak{r}_{G|_p} = \mathfrak{r}_{G|_{E(G|_p)}}$, and similarly for a partitioned set $\mathfrak{C}(G) = (V, P)$ its sub-partitioned set induced by a set $p \subseteq V$ has as its set of objects p, and its parts are all the parts in P which are subsets of p. Notice that, since we assumed here that $p \in P$, this means that $\mathfrak{C}(G)|_p$ will always have p itself as a part.

That is, whenever we look at the hypergraph induced by a part of the partition, and cluster just that sub-hypergraph, we get back the same thing as if we just restricted our clustering to that part.

In the setting of non-overlapping partitions, this property is more intuitive to state: If we cluster some graph, pick out one of the parts, and cluster just the subgraph induced by that part, we get back a trivial partition into a single part. Adapting this into the overlapping clustering world unfortunately gets us a less transparent definition.

4 Representability

Definition 4.1. Given a not necessarily finite set $\mathfrak{R}$ of graphs, we define the endofunctor $\Phi_{\mathfrak{R}}$ on $\mathcal{G}$ by that, for any graph $G = (V, E)$,

1. the vertices of $\Phi_{\mathfrak{R}}(G)$ are simply V,
2. the edges of $\Phi_{\mathfrak{R}}(G)$ are given by

$$E(\Phi_{\mathfrak{R}}(G)) = \bigcup_{R \in \mathfrak{R}} \hom(R, G),$$

   the set of all morphisms from a graph in $\mathfrak{R}$ into G,
3. and the edge-assigning function $\mathfrak{r}_{\Phi_{\mathfrak{R}}(G)} : E(\Phi_{\mathfrak{R}}(G)) \to 2^V$ simply assigns each morphism to its image, i.e. for $\omega \in \hom(R, G)$, we set $\mathfrak{r}_{\Phi_{\mathfrak{R}}(G)}(\omega) = \omega_*(R)$.

On morphisms, $\Phi_{\mathfrak{R}}$ is always just the identity on set functions, so if $f : G \to H$ is a morphism, $\Phi_{\mathfrak{R}}(f) = f$ as a set function.

In other words, more concretely, we get a hyperedge in $\Phi_{\mathfrak{R}}(G)$ for every copy of a graph in $\mathfrak{R}$ inside of G.

That this definition does in fact always give us a functor is something that needs to be checked – it is not immediate from the definition, even though we said in the definition that it is an endofunctor.

Lemma 4.2. *For any representing set $\mathfrak{R}$ of graphs, $\Phi_{\mathfrak{R}}$ is indeed an endofunctor on $\mathcal{G}$.*

Proof. That it always gives a graph and that it respects identity morphisms and composition of morphisms is easy to see. The only thing we actually need to check is that if $f : H \to G$ is a morphism, then $\Phi_{\mathfrak{R}}(f) = f : \Phi_{\mathfrak{R}}(H) \to \Phi_{\mathfrak{R}}(G)$ is also a morphism.

Unwrapping this statement, what we need is that, for any two graphs $(G, E, \mathfrak{v})$ and $(G', E', \mathfrak{v}')$ and any morphism $f : G \to G'$, whenever e is an edge of $\Phi_{\mathfrak{R}}(G)$, there is an edge e' of $\Phi_{\mathfrak{R}}(G')$ such that $f_*(\mathfrak{v}_{\Phi_{\mathfrak{R}}(G)}(e)) = \mathfrak{v}'_{\Phi_{\mathfrak{R}}(G')}(e')$.

Now, this e is itself a morphism $\omega : R \to G$ for some $R \in \mathfrak{R}$, and so if we compose e with f we get a morphism $e' = f \circ e : R \to G'$, which by definition is an edge of $\Phi_{\mathfrak{R}}(G')$. That $f_*(\mathfrak{v}_{\Phi_{\mathfrak{R}}(G)}(e)) = \mathfrak{v}'_{\Phi_{\mathfrak{R}}(G')}(e')$ is then, if we unwrap our definitions, merely the statement that $f_*(e_*(R)) = (f \circ e)_*(R)$, which is trivial. □

What is going on in the proof of Lemma 4.2 is much clearer if we just think about it in terms of things being subgraphs of each other, forgetting the data of *how* exactly they are subgraphs. Then the statement that $\Phi_{\mathfrak{R}}$ is a functor boils down to the statement that if ω is a subgraph of H and H is a subgraph of G, then ω is a subgraph of G, and so $\Phi_{\mathfrak{R}}(G)$ must have all the edges $\Phi_{\mathfrak{R}}(H)$ has, since edges in $\Phi_{\mathfrak{R}}(G)$ represent the presence of a certain subgraph in G.

When constructing this theory for the case of non-overlapping partitions, the crucial step is to take the connected components of the graph $F_\Omega(G)$, and so get a functor taking G to a set partition. By generalizing the notion of connected components a bit, we get many more options that will output also overlapping partitions.

Definition 4.3. The functor $\Pi : \mathcal{G}_s \to \mathcal{P}_{\text{no}}$ which sends a simple graph to its partition into connected components is called the connected components functor.

Checking that this is indeed a functor is an easy exercise, and so we omit the proof – it essentially boils down to the statement that adding an edge to a graph cannot disconnect a connected component, or equivalently that connectedness is a monotone increasing property in the edges.

In order to generalize this notion to hypergraphs, let us notice that for simple graphs, taking the line graph of the graph and looking at its connected components changes nothing – the set of vertices incident to the edges of a connected component of the line graph is a connected component of the original graph.

For hypergraphs, we get more than one possible notion of line graph, and therefore we get more than one notion of connected components.

Definition 4.4. For each $k \geq 1$, we define the k-line graph functor $\Lambda_k : \mathcal{G} \to \mathcal{G}_s$ as follows: For each $G = (V, E, \mathfrak{v})$:

1. The vertices of $\Lambda_k(G)$ are the vertex sets of the edges of G. That is,

$$E(\Lambda_k(G)) = \{\mathfrak{v}(e) \mid e \in E\}.$$

   Notice that this means that if there are multiple edges in E with the same vertex set, we still only get one vertex of $\Lambda_k(G)$ corresponding to them.

2. there is an edge $uv \in E(\Lambda_k(G))$ whenever $|u \cap v| \geq k$, that is, if the two edges of G overlap in at least k vertices.

We define, for a morphism $f : G \to H$ in $\mathcal{G}$, that $\Lambda_k(f) = f_* : 2^{V(G)} \to 2^{V(G)}$. This does at least send objects of the right type to objects of the right type, because the vertices of $\Lambda_k(G)$ and $\Lambda_k(H)$ are by construction sets of vertices of G and H respectively.

However, since $E(\Lambda_k(H))$ does not contain *every* subset of $V(H)$, we do need to check that for a vertex v of $\Lambda_k(G)$, $f_*(v)$ is indeed a vertex of $\Lambda_k(H)$. To see this, note that this v is by construction the vertex set of some edge $e \in E(G)$, and since f is a morphism, there exists some edge $e' \in E(H)$ such that $f_*(\mathfrak{V}_G(e)) = \mathfrak{V}_H(e')$ – and this $\mathfrak{V}_H(e')$ is a vertex of $\Lambda_k(H)$.

That this line graph functor is in fact a functor definitely requires a proof, since the statement is far from obvious.

Lemma 4.5. *For each k, Λ_k is in fact a functor from $\mathcal{G}$ to $\mathcal{G}_s$.*

Proof. That $\Lambda_k(G)$ is indeed a simple graph for each $G \in \mathcal{G}$ is clear, so suppose $f : G \to H$ is a morphism in $\mathcal{G}$. We need to check that $\Lambda_k(f) : \Lambda_k(G) \to \Lambda_k(H)$ is a morphism in $\mathcal{G}_s$.

So, to check this, there are two things we need to show: That $\Lambda_k(f)$ is injective as a set function, and that, for each each edge $uv \in E(\Lambda_k(G))$, there is an edge $f(u)f(v)$ in $E(\Lambda_k(H))$.

If we unwrap our definitions a bit, what we end up with is that this boils down to the following two statements, both of which are easily verified facts about the image of functions:

1. Whenever f is an injective function, f_* is as well.
2. If f is injective, then $|a \cap b| = |f_*(a) \cap f_*(b)|$.

That Λ_k sends identity morphisms to identity morphisms and respects composition of morphisms is easily seen from the corresponding statements about images of functions. □

So, now we have the line graph functor, which gives us a simple graph which we can take connected components of. This nearly gets us where we want to be – except of course the connected components of the line graph of G will be a partitioning of the vertex sets of the edges of G, not a partitioning of the vertices of G. So our last step to getting a generalized connected components functor is to turn this into a partitioning of the vertices.

Definition 4.6. The part-union operation $\Upsilon : \mathcal{P} \to \mathcal{P}$ is defined as follows:

For each $\mathcal{A} = (V, P) \in \mathcal{P}$, we let $\Upsilon(\mathcal{A}) = \mathcal{B} = (W, Q)$, where

1. The underlying set of $\mathcal{B}$ is given by

$$W = \bigcup_{p \in P} p,$$

that is, we view each part of P as itself being a set, and take the union of all these sets.[1]

2. and the parts of $\mathcal{B}$ are given by

$$Q = \left\{ \bigcup_{v \in p} v \;\middle|\; p \in P \right\},$$

that is, for each part of $\mathcal{A}$, we take the union of its members as sets, and declare that to be a part of $\mathcal{B}$.

[1] Jokes: For the first time in my mathematical life, the fact that all mathematical objects under ZFC are themselves sets has turned out to be useful for simplifying things.

Unfortunately, this Υ is not in general a functor, since there is no way to define what it does on morphisms between partitioned sets. However, it will still be useful to us.

Example 4.7. Add example proving that Υ cannot be made into a functor.

With all this in hand, we are finally able to define the connected components of a hypergraph.

Definition 4.8. For each k, the k-ly connected components functor $\Pi_k : \mathcal{G} \to \mathcal{P}$ is given by that, for any G, $\Pi_k(G) = (\Upsilon \circ \Pi \circ \Lambda_k)(G)$. On morphisms, $\Pi_k(f)$ is simply the same underlying set function – this is well defined, since clearly the underlying set of $\Pi_k(G)$ will be the same set as the vertex set of G.

We say that a graph is k-ly connected if $\Pi_k(G)$ has the entire vertex set of G as a part.

That this one is a functor also requires a proof, since it is defined as a composition of some things one of which is not itself a functor. However, the fact that the first two things are functors will be helpful in the proof.

Lemma 4.9. *For each k, Π_k is a functor.*

Proof. As usual, once we have checked that Π_k does send morphisms to morphisms, it is trivial that it respects identity morphisms and composition.

So, suppose $G = (V, E)$ and $H = (W, F)$ are two graphs, and $f : V \to W$ is a morphism from G to H. Let $E' = \{\mathfrak{V}(e) \mid e \in E\}$ and F' similarly be the set of vertex sets of H.

Now let $\mathcal{A} = (E', P) = \Pi(\Lambda_k(G))$, $\mathcal{B} = (F', P') = \Pi(\Lambda_k(G))$ be the connected components of the k-line graphs of G and H respectively. Further let $\tilde{\mathcal{A}} = (V, Q) = \Upsilon(\mathcal{A})$ and $\tilde{\mathcal{B}} = (W, Q') = \Upsilon(\mathcal{B})$.

What we need to show is that f is a morphism from $\Upsilon(\mathcal{A})$ to $\Upsilon(\mathcal{B})$ in $\mathcal{P}$. This concretely means that we need to show that, for each $q \in Q$, there is a $q' \in Q'$ such that $f_*(q) \subseteq q'$.

Now, for each $q \in Q$, there is a part $p \in P$ such that $q = \bigcup_{e \in p} e$. By functoriality of Λ_k and Π, we already know that f_* is a morphism from $\mathcal{A}$ to $\mathcal{B}$. This means that there has to exist a $p' \in P'$ such that $(f_*)_*(p) \subseteq p'$.

Now, by definition

$$(f_*)_*(p) = \{f_*(e) \mid e \in p\},$$

so if we let $q' = \bigcup_{e \in p'} e$, this must lie in Q' by construction, and we get that

$$f_*(q) = f_*\left(\bigcup_{e \in p} e\right) = \bigcup_{e \in p} f_*(e) = \bigcup_{e' \in (f_*)_*(p)} e' \subseteq \bigcup_{e' \in p'} e' = q',$$

as desired. □

Definition 4.10. A clustering scheme $\mathfrak{C}$ is *representable* if $\mathfrak{C} = \Pi_k \circ \Phi_{\mathfrak{R}}$ for some representable endofunctor $\Phi_{\mathfrak{R}}$ and some $k \geq 1$. Notice that this means that all representable clustering schemes are functorial, being the composition of two functors.

Given a set $\mathfrak{R}$ of graphs and an integer k, we write $\Pi_{\mathfrak{R},k}$ for the representable clustering scheme induced by $(\mathfrak{R}, k)$, that is,

$$\Pi_{\mathfrak{R},k} = \Pi_k \circ \Phi_{\mathfrak{R}} = \Upsilon \circ \Pi \circ \Lambda_k \circ \Phi_{\mathfrak{R}}.$$

Example 4.11. If we restrict to considering only simple graphs, it is very easy to see that the connected components functor Π is representable, since we can simply take $\mathfrak{R} = \{K_2\}$ and $k = 1$ – then $\Phi_{\mathfrak{R}}$ just sends a graph to itself, and of course connected components of the 1-line graph of a simple graph and connected components of the graph itself are the same thing.

For hypergraphs, we need to take $\mathfrak{R} = \{E_n \mid n \in \mathbb{N}\}$, where E_n is the hypergraph with n vertices and a single edge containing all the vertices. Then all $\Phi_{\mathfrak{R}}$ will do to a graph is to replace each edge with k vertices with $k!$ copies of itself, which does not change the set of vertex sets of edges, and so again we recover the usual notion of connected components by looking at the connected components of the 1-line graph of the graph.

In fact any set $\mathfrak{R}$ of connected graphs containing E_n for each n will represent Π_1 – but note that they will in general give inequivalent endofunctors on $\mathcal{G}$.

One somewhat surprising fact is that there are actually representable clustering schemes that are not finitely representable.

Theorem 4.12. *There exists a representable clustering scheme $\Pi_{\mathfrak{R},1}$ which is not* finitely *representable, that is, it is not equal to $\Pi_{\mathfrak{G},1}$ for any finite set of graphs $\mathfrak{G}$.*

A fortiori, there exists a representable endofunctor that is not finitely representable.

In order to give a proof of this, we first need a structural result about how these endofunctors behave. Let us say that an edge in a hypergraph which contains all vertices is a *spanning edge*, and a hypergraph which has a spanning edge is *spanned*.

Remark 4.13. For any $R \in \mathfrak{R}$, it is trivial to see that $\Phi_{\mathfrak{R}}(R)$ must be spanned – specifically we can take the identity morphism on R as the spanning edge.

In fact, which graphs are sent to spanned graphs by $\Phi_{\mathfrak{R}}$ essentially determines what it does as a functor. Similarly, if we are interested in the weaker equivalence of inducing the same clustering scheme, this is determined by which graphs are sent to a connected graph.

Lemma 4.14. *For any set $\mathfrak{R}$ of graphs and any graph $G = (V, E, \mathfrak{r})$, it holds that $\Phi_{\mathfrak{R}\cup\{G\}} = \Phi_{\mathfrak{R}}$ if and only if G is sent to a spanned graph by $\Phi_{\mathfrak{R}}$.*

Proof. Suppose $\Phi_{\mathfrak{R}\cup\{G\}} = \Phi_{\mathfrak{R}}$. That $\Phi_{\mathfrak{R}\cup\{G\}}(G)$ is spanned is immediate by Remark 4.13, and so since $\Phi_{\mathfrak{R}\cup\{G\}} = \Phi_{\mathfrak{R}}$, $\Phi_{\mathfrak{R}}(G)$ is spanned, as desired.

Now suppose $\Phi_{\mathfrak{R}}(G)$ is spanned, and let H be some arbitrary simple graph. We wish to show that $\Phi_{\mathfrak{R}\cup\{G\}}(H) = \Phi_{\mathfrak{R}}(H)$.

That edges of $\Phi_{\mathfrak{R}}(H)$ must also be edges of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ is obvious, so all we need to show is that edges of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ are also edges of $\Phi_{\mathfrak{R}}(H)$. Thus, suppose e is an edge of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ – by construction, this edge is a morphism ω from some $R \in \mathfrak{R} \cup \{G\}$ into H, whose image is the vertex set of this edge.

If this R is not G, then we are done. If it is G, we recall that $\Phi_{\mathfrak{R}}(G)$ is spanned, and so we can take e' to be a spanning edge of $\Phi_{\mathfrak{R}}(G)$. Again, this edge is by construction a morphism ω' from some R' – this time an $R' \in \mathfrak{R}$ – into G, whose image, since it is spanning, must be the entirety of G.

So, if we compose $\omega' : R' \to G$ and $\omega : G \to H$, we get a morphism from R' into H, which by definition is an edge of $\Phi_{\mathfrak{R}}(H)$, and since ω' was onto, the image of this composition must be the same as the image of ω, and so this edge has the right vertex set, and we are done. □

Remark 4.15. Rephrase lemma according to the following?: When one reads the above proof one can notice that the ω' we found must in fact have been an isomorphism, since it was onto, and all morphisms are assumed injective, and so it was a bijective morphism, which in this category must be an isomorphism.

Lemma 4.16. *For any set of graphs $\mathfrak{R}$ and any graph G, if $\Pi_{\mathfrak{R},k} = \Pi_{\mathfrak{R}\cup\{G\},k}$, then $\Phi_{\mathfrak{R}}(G)$ is k-ly connected. The reverse implication holds only if $k = 1$.*

Proof. Suppose $\Pi_{\mathfrak{R},k} = \Pi_{\mathfrak{R}\cup\{G\},k}$. We know from Remark 4.13 that $\Phi_{\mathfrak{R}\cup\{G\}}(G)$ is spanned, which immediately gives that it is k-ly connected. So since $\Pi_{\mathfrak{R},k} = \Pi_{\mathfrak{R}\cup\{G\},k}$, $\Phi_{\mathfrak{R}\cup\{G\}}(G)$ and $\Phi_{\mathfrak{R}}(G)$ have to have the same k-ly connected components, and therefore $\Phi_{\mathfrak{R}}(G)$ is also k-ly connected, as desired.

In the other direction, for the $k = 1$ case, suppose $\Phi_{\mathfrak{R}}(G)$ is 1-ly connected. This means $\Upsilon(\Pi(\Lambda_1(\Phi_{\mathfrak{R}}(G))))$ has a part containing every vertex, and by definition of Υ and Π, this means there is some connected component p of $\Lambda_1(\Phi_{\mathfrak{R}}(G))$ whose edges together contain every vertex. Let us call these edges $e_1, e_2, \ldots, e_n$, and observe that for each of these edges there exists an $R_i \in \mathfrak{R}$ and an $\omega_i : R_i \to G$, where $\mathrm{im}(\omega_i) = \mathfrak{V}(e_i)$.

So let H be some arbitrary graph, for which we wish to show that $\Pi_{\mathfrak{R},1}(H) = \Pi_{\mathfrak{R}\cup\{G\},1}(H)$. It is easily seen that $\Phi_{\mathfrak{R}}(H)$ is a subgraph of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$, so it will suffice to show that whenever there is a path in the line graph of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ between two edges e, e', there is a path between them also in $\Phi_{\mathfrak{R}}(H)$.

To lessen the potential confusion of terminology, we will from now on refer to the vertices of the line graph as "edges" and avoid using the word edge for the edges of the line graph, only referring to paths in the line graph and to edges (i.e. line-graph-vertices) being adjacent in such paths. The word "vertex", similarly, refers exclusively to a vertex of H.

So suppose we have such edges e, e' and a path between them in $\Lambda_1(\Phi_{\mathfrak{R}\cup\{G\}}(H))$. If this path does not contain any edge induced by a copy of G in H, then the same path exists also in $\Lambda_1(\Phi_{\mathfrak{R}}(H))$, and so we are done.

So, assume f is such an edge on this path, and say $\omega : G \to H$ is the morphism creating this edge. Let e_p be the edge previous to f on this path, and e_s be the edge subsequent to f. Since these edges are adjacent, there has to exist $v_p \in \mathfrak{V}(e_p) \cap \mathfrak{V}(f)$ and $v_s \in \mathfrak{V}(f) \cap \mathfrak{V}(e_s)$.

Now, since f arises as a copy of G in H, and the line graph of G has a connected component covering it, whose members are the edges e_i, the following has to be true: There exists $i_1, i_2, \ldots, i_m$ such that $v_p \in \omega(\mathfrak{V}(e_{i_1}))$, $v_s \in \omega(\mathfrak{V}(e_{i_m}))$, and $e_{i_1}, e_{i_2}, \ldots, e_{i_m}$ form a path in the line graph of G.

So, if we compose the morphisms ω_i with ω, we get a collection of morphisms from graphs in $\mathfrak{R}$ into H – that is, edges in $\Phi_{\mathfrak{R}}(H)$. Call these edges $e'_1, e'_2, \ldots, e'_n$. Since $v_p \in \mathfrak{V}(e_p)$ and $v_p \in \mathfrak{V}(e'_{i_1})$, we must have that e_p is adjacent to e'_{i_1}, and similarly e_s is adjacent to e'_{i_m}. Notice that this is where we use the fact that $k = 1$ – this is the step that fails for $k > 1$.

It is easy to see that the rest of the edges on the path $e_{i_1}, e_{i_2}, \ldots, e_{i_m}$ are also still adjacent, and so we can replace f on the path from e to e' by this longer path, getting a path that does not use f, and so is a path also in $\Phi_{\mathfrak{R}}(H)$, which is what we needed to show.

That the reverse implication fails for $k > 1$ is shown by the example in Figure draw and add figure. □

With this result in hand, we can now give a proof of the existence of a representable but not finitely representable endofunctor.

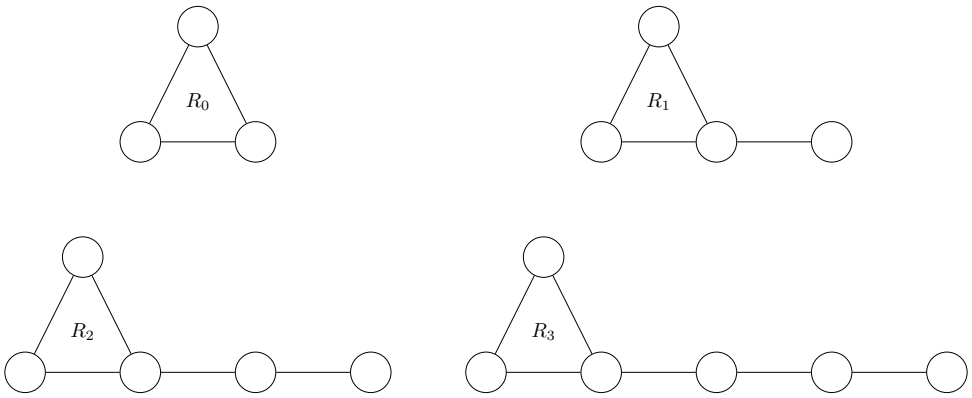


Figure 2: The family of graphs $\{R_i\}_{i=0}^3$.

Proof of Theorem 4.12. Let, for each i, R_i be the simple graph which consists of a triangle with a tail of i vertices, as shown in Figure 2, and let $\mathfrak{R} = \{R_i \mid i \in \mathbb{N}\}$. We claim that $\Pi_{\mathfrak{R},1}$ is not equal to $\Pi_{\mathfrak{G},1}$ for any finite $\mathfrak{G}$.

To prove this, let us first define

$$\bar{\mathfrak{R}} = \{G \in \mathcal{G} \mid \Phi_{\mathfrak{R}}(G) \text{ is 1-ly connected.}\}$$

as the set of all graphs which $\Phi_{\mathfrak{R}}$ sends to 1-ly connected graphs. By Lemma 4.16 respectively, $\Pi_{\bar{\mathfrak{R}},1} = \Pi_{\mathfrak{R},1}$, and $\bar{\mathfrak{R}}$ is the greatest set with this property.

It is not hard to see that any graph in $\bar{\mathfrak{R}}$ has to contain a triangle, since $\Phi_{\mathfrak{R}}$ will send any triangle-free graph to a graph with no edges, which is obviously not connected.

Now suppose for contradiction that $\mathfrak{G}$ is a finite set of graphs such that $\Phi_{\mathfrak{G}} = \Phi_{\bar{\mathfrak{R}}}$. Lemma 4.16 implies that we must have $\mathfrak{G} \subset \bar{\mathfrak{R}}$, since if there were a $G \in \mathfrak{G} \setminus \bar{\mathfrak{R}}$ it'd be sent to a 1-ly connected graph by $\Phi_{\mathfrak{G}}$, and thus also by $\Phi_{\bar{\mathfrak{R}}}$, since we've assumed $\Phi_{\mathfrak{G}} = \Phi_{\bar{\mathfrak{R}}}$ – but we chose $\bar{\mathfrak{R}}$ so that it already contains everything sent to a connected graph by $\Phi_{\bar{\mathfrak{R}}}$.

So, let

$$r = \max_{G \in \mathfrak{G}} \max_{u \in V(G)} \min_{\substack{v \in V(G) \\ v \text{ is in a triangle}}} d(u, v).$$

That is, r is the furthest any vertex in any graph in $\mathfrak{G}$ is from a triangle. We claim that $\Phi_{\mathfrak{G}}$ will not send R_{r+1} to a connected graph, proving it is not equal to $\Phi_{\bar{\mathfrak{R}}}$, giving our contradiction.

That this is so is actually easy to see – if R_{r+1} were sent to a connected graph, that must in particular mean there is an edge in $\Phi_{\mathfrak{G}}(R_{r+1})$ between the tail vertex and its neighbour. So there is some morphism from some $G \in \mathfrak{G}$ that hits both vertices. However, such a morphism must also map the triangle in G onto the triangle in R_{r+1}, since that is the only place the triangle can go. Then, however, it cannot also reach to the tail – the tail is further away from the triangle than any vertex in any G is from the triangle. Thus, no such morphism can exist, and we are done. This paragraph needs a bit more checking that everything still works in the hypergraph case. □

Remark 4.17. The construction in our proof of course works equally as well replacing the

triangle by any connected graph that is not a line. So we have found a large family of examples of classes of graphs that give representable but not finitely representable endofunctors.

This gives rise to a more general structural question. If we say that a graph class $\mathcal{C}$ is *representable* if there exists some $\mathfrak{R}$ such that $\bar{\mathfrak{R}} = \mathcal{C}$, we can ask ourselves which classes of graph are representable, and which are finitely representable. Our proof above essentially relied on showing that the class of connected triangle-free graphs is representable, but not finitely representable.

We have shown before that the connected simple graphs are representable, with $\mathfrak{R} = \{K_2\}$, and a similar construction will get the classes of connected hypergraphs of different uniformities. The example we gave in the proof easily generalises to showing that the graphs of girth at most k are representable but not finitely representable, by extending from triangles with tails to all cycles of length at most k with tails.

The class of non-planar graphs is also representable, with itself as a representation, since there can of course be no morphism from a non-planar graph into a planar graph. Whether there exists a finite representing set is a potentially interesting question, which we leave unanswered.

5 Equivalence of representability and excisiveness

We are able to exactly characterise the relationship between the three concepts of functoriality, representability, and excisiveness for clustering schemes.

Theorem 5.1. *A clustering scheme is representable if and only if it is excisive and functorial. No other implications hold between these concepts.*

We divide the proof of Theorem 5.1 into two lemmas and a collection of examples, as illustrated in Figure 3. We have already seen from Lemma 4.2 that representability implies functoriality, so it remains to see that it also implies excisiveness, and then to show the necessity of the condition.

Lemma 5.2. *A representable clustering scheme is excisive.*

Proof. Let $\mathfrak{R}$ be some set of graphs, and $G = (V, E)$ be some graph. Assume $\Pi_{\mathfrak{R},k}(G) = (V, P)$, and let $p \in P$ be some part. What we need to show is that $\Pi_{\mathfrak{R},k}(G|_p) = \Pi_{\mathfrak{R},k}(G)|_p$.

Let p' be the edge set of $\Phi_{\mathfrak{R}}(G|_p)$, and let p'' be the connected component of $\Lambda_k(\Phi_{\mathfrak{R}}(G))$ which is sent to p by Υ. Notice that we have $p'' \subseteq p'$, but it can be a strict subset when $k > 1$, as demonstrated by the example in Figure draw and add example of this.

To simplify our notation, let $H = \Phi_{\mathfrak{R}}(G)$. We will show our result by proving the following statements:

1. $\Phi_{\mathfrak{R}}(G|_q) = \Phi_{\mathfrak{R}}(G)|_q = H|_q$ for any set $q \subseteq V$,
2. $\Lambda_k(H|_q) = \Lambda_k(H)|_{q'}$ for any set $q \subseteq V$, where q' is the edge set of $H|_q$,
3. $\Pi(\Lambda_k(H)|_{p''}) = \Pi(\Lambda_k(H))|_{p''}$, and
4. $\Upsilon(\Pi(\Lambda_k(H))|_{p'}) = \Upsilon(\Pi(\Lambda_k(H)))|_p$.

For the first statement, that they have the same vertex sets is obvious, and since $G|_q$ is a subgraph of G and $\Phi_{\mathfrak{R}}$ is a functor, we must have that $\Phi_{\mathfrak{R}}(G|_q)$ is a subgraph of $\Phi_{\mathfrak{R}}(G)$. So if we can show that any edge of $\Phi_{\mathfrak{R}}(G)|q$ is also an edge of $\Phi_{\mathfrak{R}}(G|_q)$, we will be done.

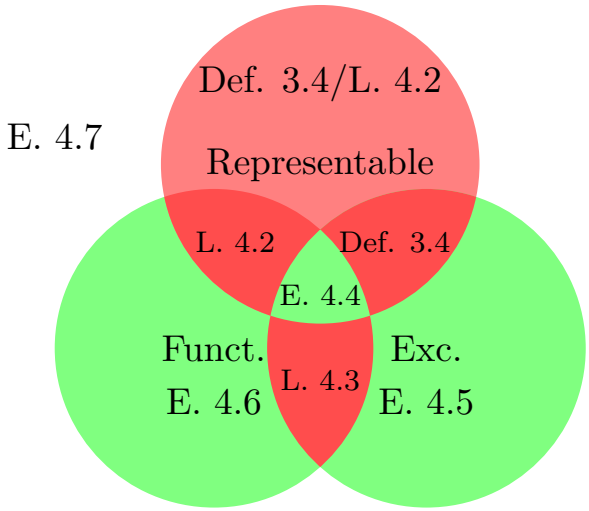


Figure 3: A Venn diagram illustrating the structure of the proof of Theorem 5.1. Impossible combinations of properties are in red, possible ones (other than having no property) are in green, and the labels indicate the lemma or example proving the intersection is empty or non-empty.

So suppose e is such an edge, and take $R \in \mathfrak{R}$ and $\omega : R \to G$ to witness this edge. We must have that $\omega(R) \subseteq q$ – but this means we can simply consider ω as a morphism into $G|_q$, and so it will also be an edge of $\Phi_{\mathfrak{R}}(G|_q)$, as needed.

For the second statement, functoriality of Λ_k immediately gives us that $\Lambda_k(H|_q)$ is a subgraph of $\Lambda_k(H)$, and by definition the vertex set of $\Lambda_k(H|_q)$ is q'. So again, what we need to show is that any edge of $\Lambda_k(H)|_{q'}$ is already an edge of $\Lambda_k(H|_q)$. Suppose we have two vertices $e, f \in q'$ which are adjacent – that is, $|e \cap f| \geq k$, since a vertex of the line graph is a vertex set of an edge of the underlying graph.

Now, clearly, that $e, f \in q'$ implies that $e, f \subseteq q$, and so their intersection also lies in q. Thus they also overlap in at least k vertices also seen as vertices of $H|_q$, and so they are adjacent in the line graph of $H|_q$, as desired.

The third statement is a nearly trivial statement about the connected components functor – if we take the subgraph of a simple graph induced by one of its connected components, and then take the connected components of that, we get back just the connected component we started with. □

Lemma 5.3. *An excisive and functorial clustering scheme is representable.*

Proof. Let $\mathfrak{C}$ be some excisive and functorial clustering scheme, and let

$$\mathfrak{R} = \{G \in \mathcal{G} \mid \mathfrak{C}(G) = (V_G, \sim_{\text{cpl}})\},$$

that is, $\mathfrak{R}$ is the set of all graphs clustered by $\mathfrak{C}$ into a single part. We will show that in fact $\mathfrak{C} = \Phi_{\mathfrak{R}}$.

So, let G be some graph. We need to show two things:

1. Whenever u and v are clustered in the same part by $\mathfrak{C}$, they are in the same connected component of $\Phi_{\mathfrak{R}}(G)$.

2. Whenever uv is an edge of $\Phi_{\mathfrak{R}}(G)$, u and v are clustered in the same part by $\mathfrak{C}$. This of course implies that whenever u and v are in the same connected component, they are also clustered in the same part by $\mathfrak{C}$, which is what we really need.

We start with the first direction, letting u and v be two vertices that are sent to the same part by $\mathfrak{C}$ – call this part A, and let $H = G|_A$. By excisiveness of $\mathfrak{C}$, we have that $\mathfrak{C}$ sends H to a single part, so $H \in \mathfrak{R}$. Therefore, the inclusion morphism $f : H \hookrightarrow G$ is a morphism from something in $\mathfrak{R}$ that hits both u and v, showing there is an edge uv in $\Phi_{\mathfrak{R}}(G)$, as desired.

In the second direction, assume that uv is an edge of $\Phi_{\mathfrak{R}}(G)$. Thus, there is some $\omega \in \mathfrak{R}$ and $f_\omega : \omega \to G$ such that $\mathrm{im}(f_\omega) \ni u, v$. By functoriality of $\mathfrak{C}$, f will also be a morphism from $\mathfrak{C}(\omega)$ into $\mathfrak{C}(G)$ – which by how we defined morphisms in the category of partitioned sets means that any two vertices equivalent in $\mathfrak{C}(\omega)$ must also be equivalent in $\mathfrak{C}(G)$. However, we defined $\mathfrak{R}$ as precisely the set of things such that $\mathfrak{C}(\omega)$ has only a single part – so everything is equivalent in $\mathfrak{C}(\omega)$ and in particular $u \sim_{\mathfrak{C}(\mathfrak{R})} v$, and so they are also equivalent in $\mathfrak{C}(G)$, as desired. □

Finally, in order to show that there is no other implication that holds between these concepts, it suffices to give examples of clustering schemes with every combination of properties not already ruled out.

Example 5.4. The connected components functor Π is representable and excisive – we can take its representation to be $\mathfrak{R} = \{K_2\}$.

Example 5.5. For a scheme that is excisive but not functorial (and thus not representable), consider the scheme that partitions all graphs into a single part except K_2, which it partitions into two parts.

Example 5.6. For a scheme which is functorial but neither excisive nor representable, cluster all connected components on more than two vertices as well as all isolated vertices as their own parts.

If there is exactly one component on two vertices, i.e. an isolated edge, cluster it as two parts, otherwise cluster all isolated edges as single parts.

Example 5.7. Finally, for a scheme which has none of the properties, cluster all graphs as a single part except for K_2, which we cluster as two parts, and the disjoint union of two copies of K_2, which we cluster as two parts – that is, each of the copies is its own part.

It is possible to carry out this entire argument, showing equivalence between representability and excisiveness, also in the setting where we do not require morphisms in the category of graphs to be injective. The arguments only need minor modifications to deal with this.

However, it turns out that this setting is much less interesting: There are in fact only three different representable clustering schemes.

Theorem 5.8. *If we do not require our morphisms to be injective, there are exactly three possible representable clustering schemes:*

1. *The scheme that always sends every vertex to its own part, represented by the empty set or by K_1,*

2. *the connected components functor, represented by any set of connected graphs containing at least one graph on at least two vertices,*

3. *and the scheme that always sends all vertices to the same part, represented by any set of graphs containing a disconnected graph.*

Proof. Suppose we have some set of simple graphs $\mathfrak{R}$, and some graph G. If $\mathfrak{R}$ is empty, or contains only a K_1, it is clear that there are no morphisms at all from any $\omega \in \mathfrak{R}$ that hit two vertices of G, and so there can trivially be no edges in $\Phi_{\mathfrak{R}}(G)$, and the resulting clustering scheme always sends every vertex to its own part.

Now suppose $\mathfrak{R}$ contains some graph that is not K_1, say ω, but this ω is disconnected. Then, for any two vertices $u, v \in G$, we can send one connected component of ω to u and all the other connected components to v. Thus there is an edge uv in $\Phi_{\mathfrak{R}}(G)$, and we have shown that $\Phi_{\mathfrak{R}}$ must send all graphs to cliques, and thus the clustering scheme clusters all graphs as a single part.

Finally, suppose $\mathfrak{R}$ contains some graph on at least two vertices, and every $\omega \in \mathfrak{R}$ is connected. Then, pick some $\omega \in \mathfrak{R}$ and a vertex $w_\omega \in \omega$. For any edge uv in G, we can send w_ω to u and every other vertex of ω to v. Thus, uv must also be an edge of $\Phi_{\mathfrak{R}}(G)$. That we cannot get an edge in $\Phi_{\mathfrak{R}}(G)$ between two vertices in different connected components of G is clear from that $\mathfrak{R}$ contains only connected graphs. Therefore, $\Phi_{\mathfrak{R}}(G)$ must have the same connected components as G, and so the induced clustering scheme must be just the connected components functor. □

6 Computational complexity of representable clustering schemes

As stated in the introduction, one benefit of this approach to clustering is that it yields tractable algorithms for exactly computing the partitioning. In this section, we give a proof of this.

The most naïve possible algorithm given a fixed set $\mathfrak{R}$ of representing graphs and an n-vertex graph G is to, for each $\omega \in \mathfrak{R}$ and each $H \in \binom{V(G)}{|\omega|}$, that is, each $|\omega|$-sized subset of the vertices of G, check whether $G|_H$ contains a subgraph isomorphic to ω. It is easy to see that this will give an algorithm that is polynomial in n, but with an exponent and constant depending on $\mathfrak{R}$.

What is more interesting is that we can get a quadratic time algorithm on any class of graphs of bounded expansion. Let us first recall what this means, before stating the lemma we will use.

Definition 6.1. A class of graphs $\mathcal{C}$ is said to have *bounded expansion* if for every $t \in \mathbb{N}$ there exists a c_t such that, whenever H is a shallow minor of depth t of some graph in $\mathcal{C}$, it holds that

$$\frac{|E(H)|}{|V(H)|} \leq c(t).$$

This notion generalizes both proper minor closed classes and classes of bounded degree, and can equivalently be defined in terms of low tree-width decompositions.[NOW12]

To get our quadratic time algorithm, we will use the following result from [ND12, Corollary 18.2, p. 407]:

Lemma 6.2. *Let $\mathcal{C}$ be a class with bounded expansion and let H be a fixed graph. Then there exists a linear time algorithm which computes, from a pair (G, S) formed by a graph $G \in \mathcal{C}$ and a subset S of vertices of G, the number of isomorphs of H in G that include some vertex in S. There also exists an algorithm running in time $O(n) + O(k)$ listing all such isomorphs, where k denotes the number of isomorphs.*

Theorem 6.3. *Let $\mathfrak{R}$ be any finite set of simple graphs, and $\Phi_{\mathfrak{R}}$ the representable clustering scheme represented by it. For any class $\mathcal{C}$ with bounded expansion, there exists an algorithm that given any graph $G \in \mathcal{C}$ and vertex v of G computes the part containing v of $\Phi_{\mathfrak{R}}(G)$ running in time $O\left(n^2\right) + O(w)$, where w is the number of subgraphs of G isomorphic to a graph in $\mathfrak{R}$.*

Proof. We will of course use the algorithm from Lemma 6.2, and we do so in the most straightforward way.

Algorithm 1 Computation of part of v in G

$P_0 \leftarrow \{v\}, P_{-1} \leftarrow \emptyset$
for $i = 1, 2, \ldots, n$ **do**
  if $P_{i-1} \setminus P_{i-2} \neq \emptyset$ **then**
    for $\omega \in \mathfrak{R}$ **do**
      Run the algorithm of Lemma 6.2 with input $(G, P_{i-1} \setminus P_{i-2})$, getting a list of isomorphs $\gamma_{\omega,1}, \ldots, \gamma_{\omega,k_\omega}$ of ω.
    end for
    $P_i \leftarrow P_{i-1} \cup \bigcup_{\omega \in \mathfrak{R}} \bigcup_{j=1}^{k_\omega} V(\gamma_{\omega,j})$
  else
    return P_{i-1}
  end if
end for
return P_n

In the absolute worst case, each time-step of the algorithm discovers exactly one new vertex, so the loop can take at most n steps, and each step takes time $O(n)$ plus some time linear in the number of isomorphs found. It is not too hard to see that each isomorph will only be found in exactly two steps, so the sum of this term over all time steps will be linear in the total number of isomorphs. □

7 Representability for hierarchical clustering

Definition 7.1. A functorial hierarchical clustering scheme is a functor from the product category $\mathcal{G} \times \mathbb{R}^+$ into $\mathcal{P}$, where we make $\mathbb{R}^+ = [0, \infty)$ into a category by saying there is a morphism from a to b whenever $a \geq b$.

There is actually a natural extension of our definition of a representable clustering scheme also to this setting, but it requires that we set up some more machinery.

Definition 7.2. The category $\mathcal{G}^w$ of weighted simple graphs has as its objects simple graphs together with a function assigning a real-valued positive weight to each edge of the graph. A morphism from (V, E, w) to (V', E', w') is an injective set function $f : V \to V'$ such that whenever uEv, $f(u)E'f(v)$ and $w(uv) \leq w'(f(u)f(v))$.

Definition 7.3. The scissors functor Σ from $\mathcal{G}^w \times \mathbb{R}^+$ into $\mathcal{G}$ takes in a weighted graph $G = (V, E, w)$ and a non-negative real number τ, removes all edges whose weight is below τ, and then forgets all the remaining weights, getting just a simple graph.

Lemma 7.4. *The scissors functor is actually a functor.*

Proof. Similarly to how we proved that the representable endofunctors are in fact functors, most of the required properties are obvious. There are only two things we need to check:

1. If f is a morphism of weighted graphs sending $G = (V, E, w)$ to $G' = (V', E', w')$, then for every $\tau \in \mathbb{R}^+$, f is also a morphism in $\mathcal{G}$ from $\Sigma(G, \tau)$ to $\Sigma(G', \tau)$,
2. and if $\alpha \geq \beta \geq 0$, then for any weighted graph $G = (V, E, w)$, the identity function $\mathrm{id} : V \to V$ is a morphism from $\Sigma(G, \alpha)$ to $\Sigma(G, \beta)$.

We start with the first item, and suppose that $f : G \to G'$ is a morphism of weighted graphs, and $\tau \geq 0$ some threshold. We wish to show that whenever uv is an edge of $\Sigma(G, \tau)$, $f(u)f(v)$ is an edge of $\Sigma(G', \tau)$. That uv is an edge of $\Sigma(G, \tau)$ means that uv is an edge of G with weight at least τ, and since f is a morphism of weighted graphs, $f(u)f(v)$ must be an edge of G', and since such morphisms are weight non-decreasing, its weight must still be at least τ. Thus it is an edge of $\Sigma(G', \tau)$ as well.

The second item is even more trivial than the first – if we unwrap the definitions, all it says is that if we lower the threshold for when edges are removed, then we will not lose any edges. □

Definition 7.5. A representable functor $\Phi_{\mathfrak{R}}$ from $\mathcal{G}$ to $\mathcal{G}^w$ is determined by a set $\mathfrak{R}$ of pairs (ω, w) of simple graphs ω and positive weights w. For any graph G, there is an edge uv of $\Phi_{\mathfrak{R}}(G)$ if there is some subgraph of G containing u and v which is isomorphic to some graph in $\mathfrak{R}$. The weight of this edge is then given by

$$w(uv) = \sum_{(\omega, w) \in \mathfrak{R}} \sum_{f \in \mathrm{Hom}(\omega, G)} \frac{w}{|\mathrm{Aut}(\omega)|} \mathbb{1}_{u,v \in \mathrm{im}(f)}.$$

Intuitively, the weight $w(uv)$ counts how many subgraphs of G contain both u and v and are isomorphic to something in $\mathfrak{R}$, except the sum is weighted. We include the division by the size of the automorphism group so that we are actually counting *subgraphs*, not embeddings. One illustration of what is going on in this definition can be found in Figure 4.

Lemma 7.6. *The representable functors from $\mathcal{G}$ to $\mathcal{G}^w$ are actually functors.*

Proof. As before, the only thing we need to explicitly check is that whenever $f : G \to G'$ is a morphism in $\mathcal{G}$, it is also a morphism from $\Phi_{\mathfrak{R}}(G)$ to $\Phi_{\mathfrak{R}}(G')$. So, in particular, assume that uv is an edge of $\Phi_{\mathfrak{R}}(G)$ with weight α – what we need to show is that $f(u)f(v)$ is an edge of $\Phi_{\mathfrak{R}}(G')$ with weight at least α.

That it is an edge at all is easy to see – that it is an edge in $\Phi_{\mathfrak{R}}(G)$ means there is some $\omega \in \mathfrak{R}$ and a morphism $f_\omega : \omega \to G$ whose image contains both u and v. If we just compose this f_ω with f, we get a morphism from ω into G' whose image contains $f(u)$ and $f(v)$.

To show that its weight is at least α, we compute

$$\begin{aligned}
w_{\Phi_{\mathfrak{R}}(G')}(uv) &= \sum_{(\omega,w)\in\mathfrak{R}} \sum_{f'_\omega\in\mathrm{Hom}(\omega,G')} \frac{w}{|\mathrm{Aut}(\omega)|}\mathbb{1}_{f(u),f(v)\in\mathrm{im}(f'_\omega)} \\
&\geq \sum_{(\omega,w)\in\mathfrak{R}} \sum_{\substack{f'_\omega\in\mathrm{Hom}(\omega,G') \\ \exists f_\omega\in\mathrm{Hom}(\omega,G): f'_\omega = f_\omega\circ f}} \frac{w}{|\mathrm{Aut}(\omega)|}\mathbb{1}_{f(u),f(v)\in\mathrm{im}(f'_\omega)} \\
&= \sum_{(\omega,w)\in\mathfrak{R}} \sum_{f_\omega\in\mathrm{Hom}(\omega,G)} \frac{w}{|\mathrm{Aut}(\omega)|}\mathbb{1}_{f(u),f(v)\in\mathrm{im}(f_\omega\circ f)} \\
&= \sum_{(\omega,w)\in\mathfrak{R}} \sum_{f_\omega\in\mathrm{Hom}(\omega,G)} \frac{w}{|\mathrm{Aut}(\omega)|}\mathbb{1}_{u,v\in\mathrm{im}(f_\omega)} = w_{\Phi_{\mathfrak{R}}(G)}(uv) = \alpha,
\end{aligned}$$

where the inequality is just that we sum over a subset of the summands, and the second equality follows from that our morphisms are injective. □

Definition 7.7. A representable hierarchical clustering scheme is a functor from $\mathcal{G}\times\mathbb{R}^+$ into $\mathcal{P}$ that can be written as $(\Phi_{\mathfrak{R}}\times\mathrm{id})\circ\Sigma\circ\Pi$ for some representable functor $\Phi_{\mathfrak{R}}$ from $\mathcal{G}$ to $\mathcal{G}^w$.

Remark 7.8. We can turn such any hierarchical clustering scheme into a non-hierarchical clustering functor by just fixing a threshold. That is, given $F:\mathcal{G}\times\mathbb{R}^+\to\mathcal{P}$, we can pick a threshold $\tau\in[0,\infty)$, and get a clustering scheme $F_\tau:\mathcal{G}\to\mathcal{P}$ by precomposing F with the functor $T_\tau:\mathcal{G}\to\mathcal{G}\times\mathbb{R}^+$ which sends G to (G,τ).

Unfortunately, it turns out that applying this process to a representable hierarchical clustering scheme will *not* in general result in an excisive, and thus also not a representable, functor!

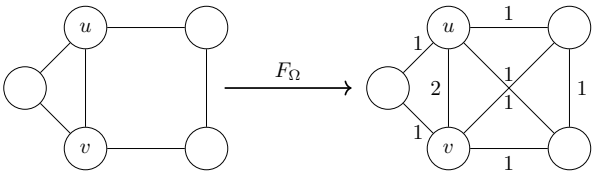


Figure 4: An example of a graph showing that $\mathfrak{R}=\{(K_3,1),(C_4,1)\}$ does not give an excisive clustering functor at threshold 1.5.

To see this, take the triangle and the four-cycle as your representing set, both with weight one, and consider what is happening in Figure 4. If we apply the scissors functor at threshold 1.5, we will keep only the edge between u and v, so they get clustered as one part. However, if we zoom in on just this part, and try to cluster this K_2, it will get clustered as two parts. Thus, what we have seen is that at this threshold, it is in fact not excisive.

Acknowledgments

We thank Professors Tatyana Turova and Svante Janson for their helpful comments on a previous version of this paper.

References

[BAB12] Ahmad Barirani, Bruno Agard, and Catherine Beaudry. "Discovering and assessing fields of expertise in nanomedicine: a patent co-citation network perspective". In: *Scientometrics* 94.3 (Nov. 2012), pp. 1111–1136. DOI: 10.1007/s11192-012-0891-6. URL: https://doi.org/10.1007/s11192-012-0891-6 (cit. on p. 1).

[Bet23] Richard F. Betzel. "Chapter 7 - Community detection in network neuroscience". In: *Connectome Analysis*. Ed. by Markus D Schirmer, Tomoki Arichi, and Ai Wern Chung. Academic Press, 2023, pp. 149–171. ISBN: 978-0-323-85280-7. DOI: https://doi.org/10.1016/B978-0-323-85280-7.00016-6. URL: https://www.sciencedirect.com/science/article/pii/B9780323852807000166 (cit. on p. 1).

[Blo+08] Vincent D. Blondel et al. "Fast unfolding of communities in large networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (Oct. 2008), P10008. DOI: 10.1088/1742-5468/2008/10/p10008. URL: https://doi.org/10.1088/1742-5468/2008/10/p10008 (cit. on p. 2).

[Bra+08] Ulrik Brandes et al. "On modularity clustering". In: *IEEE Transactions on Knowledge and Data Engineering* 20.2 (Feb. 2008), pp. 172–188. DOI: 10.1109/tkde.2007.190689. URL: https://doi.org/10.1109/tkde.2007.190689 (cit. on p. 2).

[CM13] Gunnar Carlsson and Facundo Mémoli. "Classifying clustering schemes". In: *Foundations of Computational Mathematics* 13.2 (Jan. 2013), pp. 221–252. DOI: 10.1007/s10208-012-9141-9. URL: https://doi.org/10.1007/s10208-012-9141-9 (cit. on p. 2).

[Cos+16] José M. Costa et al. "Sampling completeness in seed dispersal networks: When enough is enough". In: *Basic and Applied Ecology* 17.2 (2016), pp. 155–164. ISSN: 1439-1791. DOI: https://doi.org/10.1016/j.baae.2015.09.008. URL: https://www.sciencedirect.com/science/article/pii/S1439179115001255 (cit. on p. 1).

[CR10] Pu Chen and Sidney Redner. "Community structure of the physical review citation network". In: *Journal of Informetrics* 4.3 (2010), pp. 278–290. ISSN: 1751-1577. DOI: https://doi.org/10.1016/j.joi.2010.01.001. URL: https://www.sciencedirect.com/science/article/pii/S1751157710000027 (cit. on p. 1).

[Evk+21] Bojan Evkoski et al. "Evolution of topics and hate speech in retweet network communities". In: *Applied Network Science* 6.1 (Dec. 2021). DOI: 10.1007/s41109-021-00439-7. URL: https://doi.org/10.1007/s41109-021-00439-7 (cit. on p. 1).

[FB07] Santo Fortunato and Marc Barthélemy. "Resolution limit in community detection". In: *Proceedings of the National Academy of Sciences of the United States of America* 104.1 (Jan. 2007), pp. 36–41. DOI: 10.1073/pnas.0605965104. URL: https://doi.org/10.1073/pnas.0605965104 (cit. on p. 2).

[Jav+18] Muhammad Aqib Javed et al. "Community detection in networks: A multidisciplinary review". In: *Journal of Network and Computer Applications* 108 (2018), pp. 87–111. ISSN: 1084-8045. DOI: https://doi.org/10.1016/j.jnca.2018.02.011. URL: https://www.sciencedirect.com/science/article/pii/S1084804518300560 (cit. on p. 1).

[Koj+23] Sadamori Kojaku et al. *Network community detection via neural embeddings*. 2023. arXiv: 2306.13400 [physics.soc-ph] (cit. on p. 1).

[ND12] Jaroslav Nešetřil and Patrice Ossona De Mendez. *Sparsity*. Jan. 2012. DOI: 10.1007/978-3-642-27875-4. URL: https://doi.org/10.1007/978-3-642-27875-4 (cit. on p. 15).

[NG04] Michelle G. Newman and Michelle Girvan. "Finding and evaluating community structure in networks". In: *Physical Review E* 69.2 (Feb. 2004). DOI: 10.1103/physreve.69.026113. URL: https://doi.org/10.1103/physreve.69.026113 (cit. on p. 1).

[NOW12] Jaroslav Nešetřil, Patrice Ossona de Mendez, and David R. Wood. "Characterisations and examples of graph classes with bounded expansion". In: *European Journal of Combinatorics* 33.3 (2012). Topological and Geometric Graph Theory, pp. 350–373. ISSN: 0195-6698. DOI: https://doi.org/10.1016/j.ejc.2011.09.008. URL: https://www.sciencedirect.com/science/article/pii/S0195669811001569 (cit. on p. 15).

[Pei14] Tiago P. Peixoto. "Hierarchical block structures and High-Resolution model selection in large networks". In: *Physical Review X* 4.1 (Mar. 2014). DOI: 10.1103/physrevx.4.011047. URL: https://doi.org/10.1103/physrevx.4.011047 (cit. on p. 1).

[RAB09] Martin Rosvall, Daniel Axelsson, and Carl T. Bergstrom. "The map equation". In: *European Physical Journal-special Topics* 178.1 (Nov. 2009), pp. 13–23. DOI: 10.1140/epjst/e2010-01179-1. URL: https://doi.org/10.1140/epjst/e2010-01179-1 (cit. on p. 1).

[TWV19] Vincent Traag, Ludo Waltman, and Nees Jan Van Eck. "From Louvain to Leiden: guaranteeing well-connected communities". In: *Scientific Reports* 9.1 (Mar. 2019). DOI: 10.1038/s41598-019-41695-z. URL: https://doi.org/10.1038/s41598-019-41695-z (cit. on p. 2).

A Definitions of categorical language

For the reader of a more combinatorial bent, who might not recall all the definitions of category theory used in this paper, we include a short appendix stating them.

Definition A.1. A *category* $\mathcal{C}$ consists of a class of *objects* $\mathrm{ob}(\mathcal{C})$, and for each pair of objects $a, b \in \mathcal{C}$ a (possibly empty) class $\mathrm{Hom}(a, b)$ of *morphisms* from a to b.

We require these morphisms to compose, in the sense that for any $f \in \mathrm{Hom}(a, b)$ and $g \in \mathrm{Hom}(b, c)$ there exists an $fg \in \mathrm{Hom}(a, c)$, and this composition must be associative. There must also, for each $a \in \mathcal{C}$, be an identity morphism $\mathrm{id}_a \in \mathrm{Hom}(a, a)$, with the property that $f\mathrm{id}_a = f$ and $\mathrm{id}_a g = g$ whenever these compositions make sense.

It is worth remarking that the use of the word *class* instead of *set* is not accidental – the collection of all finite simple graphs is of course too big to be a set. However, since this is only so for "dumb reasons" – that there are class-many different labelings of the same graph – and the collection of isomorphism classes of finite simple graphs *is* a set, this distinction will never actually affect us, and we will elide it throughout the text.

Definition A.2. A *functor* F from a category $\mathcal{A}$ to a category $\mathcal{B}$ is a mapping that associates to each $a \in \mathcal{A}$ an $F(a) \in \mathcal{B}$, and that for each pair of objects $a, a' \in \mathcal{A}$ associates each morphism $f \in \mathrm{Hom}(a, a')$ to a morphism $F(f) \in \mathrm{Hom}(F(a), F(a'))$. We require this mapping to respect the identity morphisms, in the sense that $F(\mathrm{id}_a) = \mathrm{id}_{F(a)}$, and composition of morphisms, so that $F(fg) = F(f)F(g)$.

A functor from a category to itself is called an *endofunctor*.

The categories we study in this paper are all concrete, that is, their objects are sets with some extra structure on them, and the morphisms are just functions between the sets that respect the structure in an appropriate sense. Our functors never do anything strange – they all leave the underlying set unchanged, only changing what structure we have on the set, and they do "nothing" on the morphisms, that is, as set functions they are just the same function again between the same sets.

In this setting most of the requirements of a functor are trivial – of course it will behave correctly with identity morphisms and composition, because in a sense it isn't doing anything to the morphisms. Therefore, the one thing we need to check when proving that things are functors will be that morphisms do map to morphisms, that is, that a function that respects the structure on the sets in the first category will also respect the structure we have in the second category.

Definition A.3. Given two categories $\mathcal{A}$ and $\mathcal{B}$, the *product category* $\mathcal{A} \times \mathcal{B}$ is the category whose objects are pairs (a, b) of an object a from $\mathcal{A}$ and an object b from $\mathcal{B}$. A morphism in $\mathcal{A} \times \mathcal{B}$ from (a, b) to (a', b') is a pair of a morphism in $\mathcal{A}$ from a to a' and a morphism in $\mathcal{B}$ from b to b'.